SBD - Actividad 3.13 - Álvaro Fernández Becerra

Descarga el siguiente dataset: https://github.com/cran/RWeka/blob/master/inst/arff/weather.nominal.arff en él, los atributos predictores describen el tiempo atmosférico en un día, y el valor del atributo target indica si ese día juegan un partido al aire libre.

Entrena y valida un clasificador DecisionTreeClassifier con el dataset completo:

Es decir, hay que entrenar con todos los datos y hay que obtener las métricas con todos los datos.

Se pide hacerlo así, y no partir el dataset en uno de entrenamiento y otro de validación, porque este dataset es muy pequeño.

•Muestra la matriz de confusión.

•Muestra gráficamente el árbol de decisión.

•Muestra en un gráfico la relevancia de los atributos para el árbol de decisión generado.

```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, accuracy_score

# Cargar el archivo .arff local que ya subiste a Colab
df = pd.read_csv('weather.nominal.arff', comment='@', header=None)
```

```
df.head()
```

	0	1	2	3	4
0	sunny	hot	high	False	no
1	sunny	hot	high	True	no
2	overcast	hot	high	False	yes
3	rainy	mild	high	False	yes
4	rainy	cool	normal	False	yes

Pasos siguientes: Generar código con df New interactive sheet

```
for col in df.columns:
    df[col] = LabelEncoder().fit_transform(df[col])
```

```
df.columns
```

```
Index([0, 1, 2, 3, 4], dtype='int64')
```

```
# Target: 4a columna
x = df.drop(4, axis=1)
y = df[4]
```

- Sunny: 2
- Overcast: 0
- Rainy: 1

- hot: 1
- mild: 2
- cool: 0

- high: 0
- normal: 1

- False: 0
- True: 1

(Target)

- no: 0
- yes: 1

Clasificador entrenado con todo el dataset:

```
clf = DecisionTreeClassifier(random_state=42)
clf.fit(x, y)
```

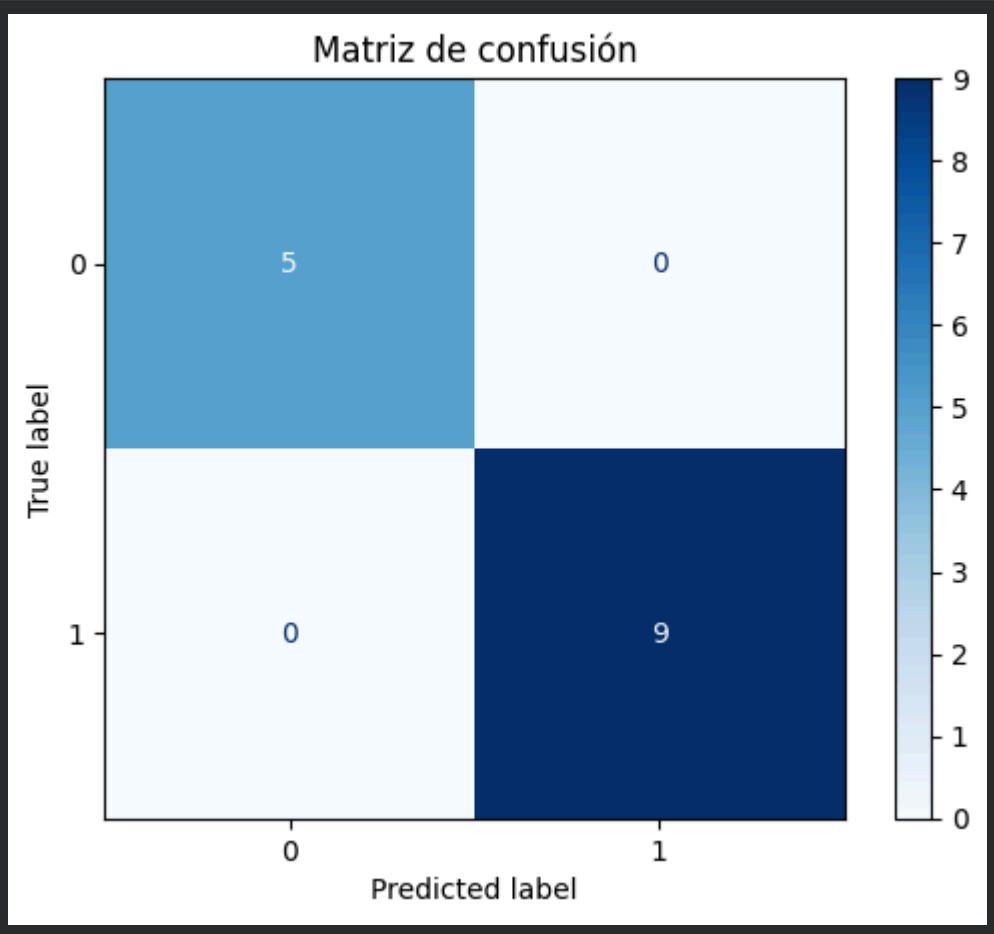
```
▾ DecisionTreeClassifier
DecisionTreeClassifier(random_state=42)
```

```
y_pred = clf.predict(x)

accuracy = accuracy_score(y, y_pred)
print(f"Accuracy: {accuracy}")
```

```
Accuracy: 1.0
```

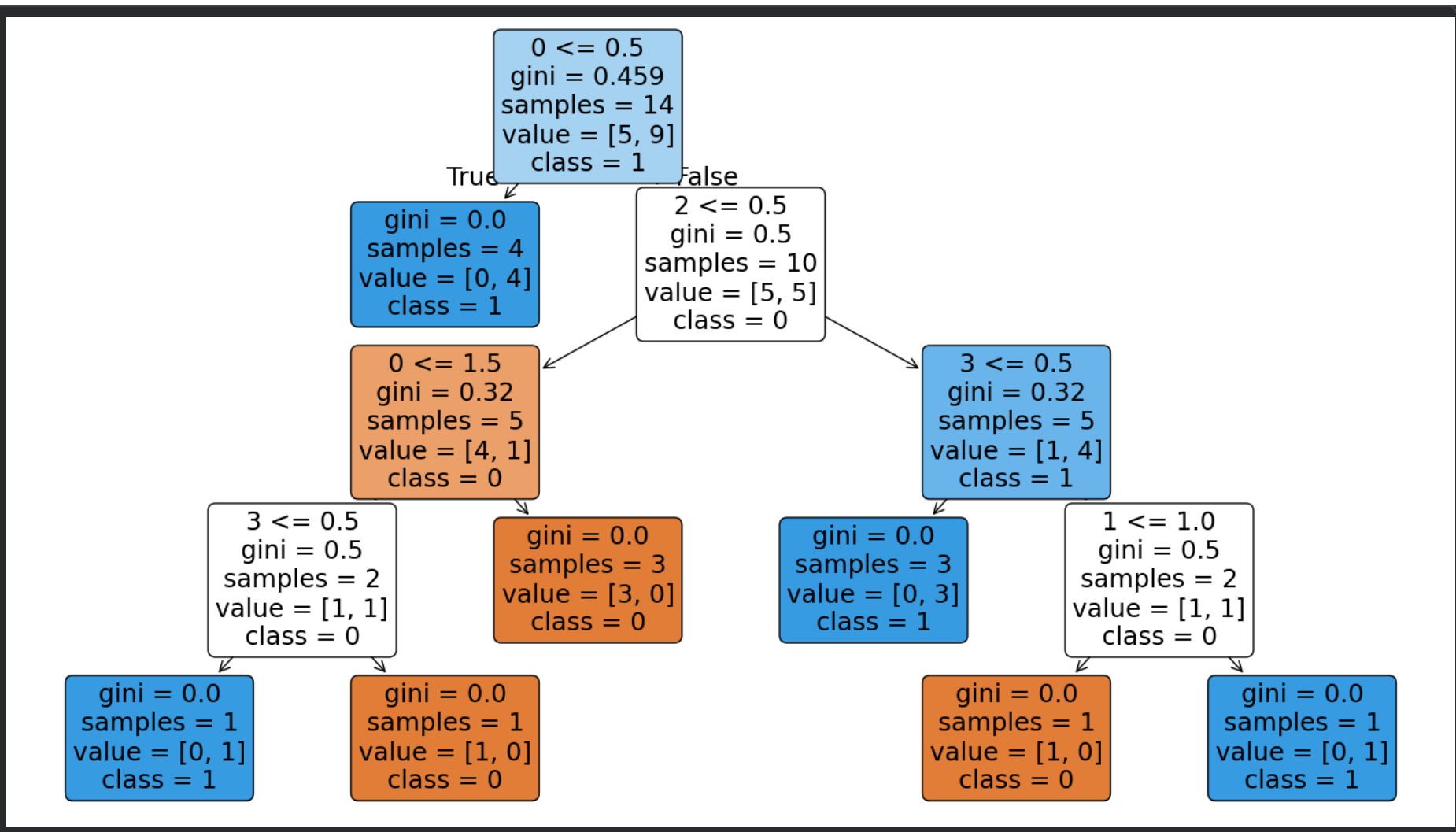
Matriz de confusion:

```
cm = confusion_matrix(y, y_pred)

disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=clf.classes_)
disp.plot(cmap='Blues')
plt.title('Matriz de confusión')
plt.show()
```



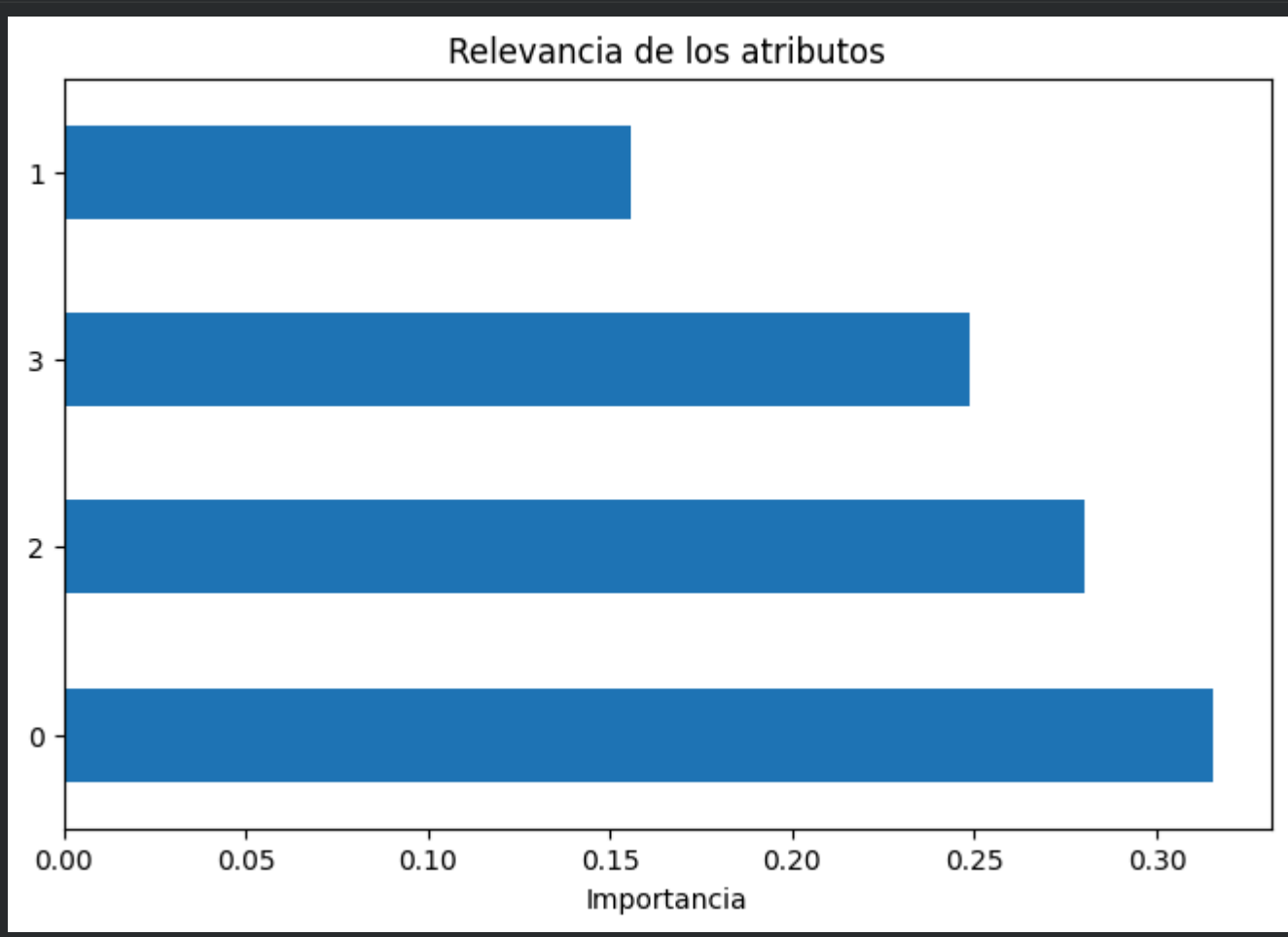
Arbol de decision:

```
plt.figure(figsize=(18, 10))
plot_tree(
    clf,
    feature_names=x.columns,
    class_names=[str(c) for c in clf.classes_],
    filled=True,
    rounded=True
)
plt.show()
```



Relevancia de atributos:

```
importancias = pd.Series(clf.feature_importances_, index=x.columns).sort_values(ascending=False)
plt.figure(figsize=(8, 5))
importancias.plot(kind='barh')
plt.title('Relevancia de los atributos')
plt.xlabel('Importancia')
plt.show()
```



- Con este grafico podemos interpretar que el atributo más relevante ha sido el 0, correspondiente al `outlook`. Es decir, si hacia dia soleado, llovia...
- El siguiente sería el atributo 2, `humidity`, lo cual tiene sentido porque si hay una alta humedad habría alta probabilidad de que lloviese y determinase si se jugaba o no.
- El siguiente sería `windy`, si hacia viento o no. Y el atributo menos relevante fue `temperature`, la temperatura.